

Are Automated Debugging Techniques really helping Programmers?

Nikhil Anand

March 25, 2026

The paper investigates whether modern automated debugging approaches genuinely improve developers’ effectiveness in practice. While a wide range of techniques—particularly those focused on fault localization through ranking suspicious code statements—have advanced the state of the art, the authors argue that these methods are often built on unrealistic assumptions about how programmers debug. In particular, many tools assume that identifying and inspecting a faulty statement in isolation is sufficient for understanding and fixing a bug. However, debugging in real-world settings is a complex, iterative process that involves navigating dependencies, examining program state, and repeatedly executing the program with different inputs. To address the gap between theoretical effectiveness and practical utility, the authors conduct human-centered empirical studies to evaluate how developers actually use automated debugging tools and whether these tools meaningfully support debugging tasks.

To explore this, the study involves controlled experiments with developers performing debugging tasks with and without a representative automated debugging tool based on statistical fault localization. The findings reveal a nuanced picture: while automated tools can sometimes help experienced developers complete simpler debugging tasks more quickly, their benefits do not consistently extend to more complex scenarios. Moreover, developers do not follow the assumptions embedded in these tools—for instance, they do not inspect ranked statements sequentially but instead rely on intuition and search strategies. The study also shows that simply pointing to a faulty statement is insufficient for bug understanding, as developers require additional context and exploration. Overall, the paper highlights a disconnect between how automated debugging techniques are designed and how debugging actually occurs in practice, emphasizing the need for tools that better align with human workflows and support richer interaction, context, and exploration.

1 Motivation

The motivation for this work stems from the well-recognized difficulty and cost associated with debugging in software development. Debugging is not only time-consuming but also cognitively demanding, often accounting for a substantial portion of software maintenance effort [1]. As modern software systems grow in complexity, configurability, and scale, the challenges associated with debugging are further amplified. This has driven significant research interest in developing automated debugging techniques aimed at improving developer efficiency and effectiveness [2, 3, 4]. Despite this progress, there remains an open question as to whether these techniques truly deliver practical benefits when used by real developers.

There exists a mismatch between how automated debugging techniques are evaluated and how debugging is actually performed in practice. Many existing approaches focus primarily on fault localization¹ - identifying and ranking potentially faulty statements—while implicitly assuming that developers can easily understand and fix a bug once the faulty statement is identified². This assumption of *perfect bug understanding* simplifies evaluation but overlooks the inherently complex and iterative nature of debugging, which involves reasoning about program behavior, exploring dependencies, and validating

¹Fault Localisation refers to identifying the lines of code responsible for the failure.

²Fault Understanding refers to understanding the root cause of the failure.

hypotheses through repeated execution [5]. As a result, current evaluation metrics may not accurately reflect the real-world usefulness of these techniques.

Secondly, a lack of empirical, human-centered studies in the debugging literature. Although foundational techniques such as program slicing have been studied for decades, only a limited number of works have investigated how developers actually use these tools in practice, and most of these studies involve small sample sizes and simplified programs [6, 7]. This scarcity of empirical evidence makes it difficult to assess whether automated debugging tools scale to realistic development scenarios or whether their underlying assumptions hold when applied to real-world tasks. Consequently, there is a need for systematic studies that examine developer behavior and tool usage in realistic debugging contexts.

Finally, the need to bridge the gap between research and practice in automated debugging. While many techniques demonstrate improvements in controlled or theoretical settings, their adoption in real development workflows remains limited. This suggests that existing tools may not align well with how developers think, explore code, and solve problems. By studying how developers interact with automated debugging tools, the authors aim to uncover the limitations of current approaches and identify directions for designing more effective, developer-centric debugging support. Ultimately, the goal is not to prove the superiority of any single technique, but to better understand what makes debugging tools useful in practice and how they can be improved to meet the needs of real users.

1.1 Automated Debugging Techniques

Early work in this area is rooted in **program slicing**, introduced by [5, 8], which identifies all statements that may affect the value of a variable at a given program point. The key idea is that if a variable exhibits incorrect behavior, the fault must lie within its slice, allowing developers to ignore irrelevant parts of the code. However, in practice, slices tend to be large and difficult to use effectively, which led to the development of more refined variants such as dynamic slicing [9] and other optimizations like relevant, critical, and pruned slices [10, 11]. While these techniques reduce the size of the candidate set, they still often produce too many statements to be practically useful.

To address the limitations of slicing-based approaches, a second class of techniques emerged that rely on analyzing program executions. **These techniques compare failing and passing runs to identify suspicious code regions, using various forms of execution data such as path profiles [12], counterexamples from model checking [3, 13], statement coverage [14], and predicate evaluations [4].** Some approaches further enhance effectiveness by applying clustering techniques to reduce redundancy in execution traces and highlight meaningful patterns [15, 16, 17]. Collectively, these methods aim to rank statements based on their likelihood of being faulty, thereby guiding developers toward potential root causes more efficiently.

Despite their sophistication, a common limitation across these techniques is their narrow focus on reducing the number of statements developers must inspect. **They generally assume that once a suspicious statement is identified, the developer can easily determine whether it is faulty and fix the issue.** This assumption overlooks the broader cognitive and investigative processes involved in debugging, such as understanding execution context, analyzing dependencies, and exploring program state. In many cases, developers must rerun the program or gather additional information to make sense of a statement’s behavior, which these techniques do not directly support.

While automated debugging techniques have significantly advanced in terms of algorithmic capability, their practical effectiveness remains uncertain. The lack of consideration for how developers actually interact with code during debugging raises concerns about their usability and real-world impact. This gap between technical design and human-centered usage motivates the need for empirical studies that evaluate these techniques in realistic settings and inform the development of more effective debugging tools.

1.2 Related Work with Programmers

Early foundational work by [5] explored how programmers interact with program slices, showing that developers are more likely to recognize meaningful code fragments when they align with execution flow.

However, these initial studies did not directly measure debugging effectiveness. Subsequent work by [7] attempted to evaluate whether slicing improves debugging performance, but found no significant benefit when developers used slicing tools on small programs. Interestingly, modifications to the experimental setup—such as using smaller programs and simplified representations like printed slices—led to some improvements, suggesting that the effectiveness of such techniques is highly sensitive to context and presentation.

Further studies investigated how developers naturally approach debugging and whether slicing aligns with their cognitive strategies. For instance, [6] found that only a small subset of programmers instinctively applied slicing-like reasoning during debugging, but those who did demonstrated better understanding and more systematic behavior. Similarly, [18] conducted controlled experiments comparing groups with and without slicing tools, finding mixed results: while some improvements in debugging time were observed for certain tasks, these benefits were not consistent across all scenarios. These findings indicate that even well-established techniques like slicing do not uniformly translate into improved debugging performance.

More recent work has explored alternative debugging aids and their impact on developer effectiveness. For example, [19] studied an Eclipse plugin that visualized differences between execution traces, finding some evidence that novice developers using the tool could outperform experts without it. Another notable system, Whyline, integrates visualization, slicing, and interactive querying to help developers understand program behavior. In a study by [20], participants using Whyline were able to debug significantly faster than those using traditional tools, demonstrating the potential of richer, more interactive debugging environments. These results suggest that combining multiple forms of support—rather than relying solely on statement ranking or slicing—may be key to improving debugging effectiveness.

Overall, the section underscores that empirical evidence for the effectiveness of automated debugging techniques remains sparse and inconclusive. Many studies involve small sample sizes, simplified programs, or limited task diversity, making it difficult to generalize findings to real-world development contexts. As a result, there is still a significant gap in understanding how these tools perform in practice and which features truly benefit developers. This lack of comprehensive evaluation motivates the need for more robust, large-scale, and realistic studies that examine both tool effectiveness and developer behavior, ultimately guiding the design of more usable and impactful debugging techniques.

2 Methodology

2.1 Program Subjects

The study uses two program subjects chosen to balance familiarity and complexity. The first subject is a Java implementation of Tetris, consisting of around 2,400 lines of code. This program represents a domain that most participants can easily understand due to its intuitive gameplay mechanics, allowing them to map program behavior to expected outcomes quickly. The second subject is NanoXML, an XML parsing library of approximately 4,400 lines of code. Unlike Tetris, NanoXML represents a less familiar and more technical domain, requiring deeper reasoning about parsing logic, namespaces, and structured data. Together, these two subjects allow the study to evaluate debugging behavior in both familiar and unfamiliar contexts.

2.2 Participants

Participants were 34 graduate students enrolled in software engineering courses, with varying levels of programming and debugging experience. Some participants had prior industry experience, including internships or full-time roles, while others primarily had academic experience. This diversity allowed the study to capture a broad spectrum of debugging behaviors, from novice to more experienced developers. Since debugging is a universal activity across experience levels, the authors considered this variation beneficial for understanding how different types of programmers interact with debugging tools.

2.3 Instruments

The primary automated debugging technique used in the study is Tarantula, a statistical fault localization method that ranks program statements based on their likelihood of being faulty. This technique was selected because it is representative of many modern debugging approaches, relies on well-established statistical principles, and has shown competitive performance in prior evaluations. Importantly, Tarantula embodies the common paradigm of ranking suspicious statements, making it suitable for investigating how developers interact with such ranked outputs.

To make the technique accessible to participants, the authors developed a custom Eclipse plugin that presents the ranked list of suspicious statements. The interface was intentionally kept simple: it displays statements along with their file names, line numbers, and suspiciousness scores, and allows users to navigate directly to code by clicking entries. Additional navigation controls enable sequential traversal of the list. This minimalistic design isolates the effect of ranking-based debugging without introducing confounding factors from more complex visualizations or interfaces.

The ranked statements used by the plugin were generated using coverage data and pass/fail information from test executions. For NanoXML, existing test suites were used, while for Tetris, the authors created a capture-replay system to record gameplay sessions as test cases. These test cases were then used to compute suspiciousness scores. The study also provided all necessary artifacts, including the plugin, subjects, and instructions, to support reproducibility and future research.

2.4 Method

The experimental method is designed to evaluate three hypotheses related to the effectiveness of automated debugging tools. Participants were divided into multiple groups and assigned debugging tasks under different conditions—some using the automated tool and others relying on traditional debugging techniques. By comparing task completion times and success rates across groups, the study assesses whether automated tools improve debugging performance and whether their effectiveness varies with task difficulty.

Additional experimental variations were introduced to examine the impact of statement ranking. In some groups, the rank of the faulty statement was artificially modified to test whether higher ranking correlates with better performance. The study also collected detailed behavioral data, including navigation logs and questionnaire responses, to analyze how developers interact with the tool, how they interpret ranked information, and what challenges they encounter. This combination of quantitative and qualitative data enables a deeper understanding of debugging behavior beyond simple performance metrics.

2.5 Procedure

The experiment was conducted in controlled environments such as classrooms or labs, with participants given time to familiarize themselves with the debugging tool beforehand. During the study, participants were assigned two debugging tasks and given a fixed time limit for each. They were required to reproduce the failure, investigate the code, identify the fault, and propose a fix while recording their progress. After completing the tasks, participants filled out a questionnaire describing their experience and strategies. This structured procedure ensured consistency across participants while capturing both performance data and subjective insights into tool usage.

3 Results

3.1 Experts are faster when using tools

The results show that automated debugging tools can improve performance, but primarily for more experienced developers and simpler tasks. For the Tetris task, participants using the tool completed the task faster on average than those using traditional debugging, although the difference was not always statistically significant across all participants. However, when focusing on high-performing participants, the improvement became significant, indicating that expertise plays a key role in leveraging such tools effectively. In contrast, for the more complex NanoXML task, no clear performance benefit was observed,

suggesting that the tool’s effectiveness is context-dependent and may not generalize across different types of debugging challenges.

3.2 The tools failed to perform harder tasks

Contrary to expectations, the automated debugging tool did not provide greater benefits for harder tasks. Although NanoXML was clearly more difficult—evidenced by lower completion rates—participants using the tool did not perform significantly better than those using traditional debugging. In fact, relative performance improvements were smaller for the harder task, indicating that the tool did not scale well with task complexity. This suggests that factors such as unfamiliarity with the codebase, increased cognitive load, or lack of contextual information may limit the usefulness of automated debugging techniques in more challenging scenarios.

3.3 Rank does not impact performance

The study examined whether the rank of the faulty statement influences debugging performance by artificially adjusting its position in the ranked list. Participants were divided into groups where the faulty statement’s rank was either preserved or modified—lowered for one task and raised for another. The expectation was that higher-ranked faults would lead to faster debugging, while lower-ranked faults would slow participants down.

The results, however, showed no significant impact of rank on performance. Even when the faulty statement was moved much higher in the list, participants did not complete the task faster. Similarly, lowering the rank did not significantly hinder performance. These findings suggest that developers do not rely strictly on ranking when navigating suspicious statements and may use other strategies to guide their debugging process.

A closer analysis revealed that participants often compensated for poor ranking by identifying related or nearby statements, particularly those within the same function or context as the fault. This indicates that ranking alone is insufficient to drive debugging effectiveness, as developers rely on broader contextual reasoning and exploration. The findings challenge the common assumption that improving rank directly translates to improved debugging performance.

3.4 Programmers search statements

The analysis of navigation logs revealed that developers do not follow a linear, top-down approach when inspecting ranked statements. Instead, they frequently jump between positions in the list, often skipping multiple entries at a time. A significant portion of navigation steps involved large jumps, indicating that developers actively search for relevant code rather than systematically inspecting each item.

Additionally, participants exhibited non-linear navigation patterns characterized by frequent changes in direction, or “zigzags,” as they moved up and down the list. Questionnaire responses confirmed that developers used intuition, hypotheses, and keyword-based reasoning to guide their exploration. These behaviors highlight that debugging is an active search process, not a passive consumption of ranked results, and that tools should better support exploratory workflows.

3.5 No Perfect Bug Understanding

The study found strong evidence against the assumption of perfect bug understanding. Even after encountering the faulty statement, most participants continued to explore other parts of the code for a significant amount of time. On average, they spent more than half of their debugging time after first seeing the fault, indicating that identifying a suspicious statement is not sufficient for understanding or fixing the bug. This underscores the importance of providing additional context, such as execution state or dependencies, to support effective debugging.

3.6 Benefits and Challenges

Participants reported that the primary benefit of the automated debugging tool was its ability to point them in the right direction rather than directly identify the fault. The tool helped narrow down the search space and suggest starting points for investigation, which participants then explored further using traditional debugging techniques such as breakpoints. This suggests that automated tools are most useful as complementary aids rather than standalone solutions.

At the same time, several challenges were identified that limited the tool’s effectiveness. Participants noted the lack of contextual information, such as runtime values, program structure, and relationships between statements, which made it difficult to interpret the ranked results. Additionally, long lists of suspicious statements and false positives led to frustration and reduced trust in the tool, causing some participants to abandon it altogether. These findings highlight the need for richer, more integrated debugging tools that provide context, explanation, and better interaction mechanisms.

4 Implications

4.1 Programmers’ Behavior

Programmers Without Tools Sometimes Fix the Failure Instead of the Fault A key observation from the study is that developers using traditional debugging approaches sometimes resolve visible symptoms of a bug rather than addressing its underlying cause. Instead of identifying and correcting the root fault, they may modify parts of the code in a way that prevents the failure from occurring, effectively masking the issue. This behavior reflects a pragmatic but potentially problematic debugging strategy, where immediate functionality is restored without ensuring long-term correctness or robustness. Such fixes may introduce hidden issues or technical debt, especially in larger systems where the root cause remains unaddressed.

Interestingly, this behavior was predominantly observed among participants who did not use the automated debugging tool. In contrast, those who used the tool were more likely to locate and fix the actual fault. This suggests that automated debugging aids can guide developers toward more principled debugging practices by narrowing their focus to relevant code regions. By highlighting suspicious statements, the tool may encourage developers to investigate deeper causes rather than applying superficial fixes. As a result, automated tools may not only improve efficiency but also promote better debugging quality and correctness.

Programmers Want Values, Overviews, and Explanations The study reveals that developers rely heavily on contextual information when debugging and that current automated tools often fail to provide sufficient support in this regard. While ranking-based tools help identify suspicious code locations, developers still turn to traditional debugging techniques—such as setting breakpoints—to inspect runtime values and understand program behavior. This indicates that fault localization alone is not enough; developers need access to dynamic information and contextual cues to interpret why a piece of code is faulty. Without this, the debugging process remains incomplete and requires additional effort outside the tool.

Furthermore, developers expressed a strong need for better organization and explanation of debugging information. They preferred higher-level overviews, such as grouping suspicious statements by methods or files, and desired explanations that connect statements to execution traces, test cases, and program slices. The lack of coherence in ranked lists makes it difficult for developers to form a mental model of the problem, reducing trust in the tool. These findings suggest that future debugging tools should move beyond simple ranking and incorporate richer, more interactive features that support exploration, explanation, and understanding of program behavior.

4.2 Threats to Validity

Time Constraints and Participant Performance One important threat to validity arises from the time limits imposed during the experiment. Participants were given a fixed duration to complete each

debugging task, which was necessary to keep the study manageable but may have influenced the results. Less experienced participants, in particular, may not have had sufficient time to fully understand the code and identify the fault, leading to incomplete or skewed performance outcomes. As a result, the findings may be biased toward more experienced or efficient participants, limiting the generalizability of the conclusions across different skill levels.

Participant Demographics and Experience Another potential threat concerns the use of graduate students as participants instead of professional developers. Although some participants had industry experience, their overall expertise and real-world exposure may not fully represent that of experienced software engineers working in production environments. Differences in debugging habits, familiarity with tools, and domain knowledge could affect how participants interact with automated debugging techniques. Consequently, the observed behaviors and results may not entirely reflect how such tools would perform in professional settings.

Task and Program Selection The choice of only two programs and specific debugging tasks introduces a limitation in terms of external validity. The selected subjects—Tetris and NanoXML—represent particular types of applications and may not capture the diversity of real-world software systems. Additionally, participants were unfamiliar with the codebases, which may have affected their ability to debug effectively compared to situations where developers work on familiar systems. Furthermore, differences in failure information, such as the presence of a stack trace in one task but not the other, could have influenced the relative effectiveness of the debugging tool. These factors limit the extent to which the results can be generalized to other programs, domains, and debugging scenarios.

5 Implications

Percentages Will Not Cut It A major finding of the study is that commonly used evaluation metrics for automated debugging—particularly percentage-based rankings—do not accurately reflect practical usefulness. Many techniques report success by showing that a faulty statement appears within a small percentage of the codebase (e.g., top 2%), which may seem promising in theory. However, in practice, developers do not inspect large portions of ranked results. Instead, they tend to focus only on the top few entries and quickly lose interest if those do not appear helpful. This creates a disconnect between reported effectiveness and actual developer behavior.

The study suggests that absolute rank is a far more meaningful metric than percentage rank. Developers care about how quickly they can reach useful information, not how small the relative search space is. Even when the rank of the faulty statement was improved significantly (e.g., from position 83 to 16), no noticeable performance improvement was observed. This indicates that beyond a certain threshold, improvements in ranking do not translate into real benefits. Therefore, future research should prioritize techniques that consistently place faults among the very top results, rather than optimizing for percentage-based metrics that may be misleading.

Focus More on Search Another important implication is that debugging tools should better support search-oriented behaviors rather than relying solely on ranking. The study found that developers naturally engage in searching and filtering when navigating suspicious statements, often using domain knowledge or hypotheses about the bug to guide their exploration. Instead of following the ranked list sequentially, they jump across entries and look for patterns or keywords that align with their expectations about the failure.

This suggests that combining ranking with search capabilities could significantly enhance debugging effectiveness. For example, allowing developers to filter statements based on relevant terms or automatically highlighting potentially useful keywords could help them quickly narrow down the search space. In some cases, search-based strategies may even be more effective than ranking alone. By incorporating mechanisms that align with how developers think and explore code, debugging tools can become more intuitive and better support real-world debugging workflows.

Grow the Ecosystem The study also highlights the importance of considering the broader debugging ecosystem rather than focusing on isolated techniques. Many automated debugging approaches require additional setup steps, such as collecting execution traces, labeling test cases, and computing suspiciousness scores. In practice, the time and effort required to perform these steps may outweigh the benefits provided by the tool, especially if the integration with existing workflows is limited.

To be truly effective, debugging tools must provide end-to-end support that seamlessly integrates all aspects of the debugging process. This includes automating data collection, managing test cases, enabling easy reruns, and supporting code inspection within a unified environment. By reducing friction and improving usability, such an ecosystem can make automated debugging tools more practical and appealing for everyday use. The findings emphasize that technical improvements alone are insufficient; attention must also be given to usability, integration, and workflow alignment.

References

- [1] I. Vessey. “Expertise in debugging computer programs”. In: *International Journal of Man-Machine Studies* 23.5 (1985), pp. 459–494.
- [2] A. Zeller and R. Hildebrandt. “Simplifying and isolating failure-inducing input”. In: *IEEE Transactions on Software Engineering*. Vol. 28. 2. 2002, pp. 183–200.
- [3] T. Ball, M. Naik, and S. K. Rajamani. “From symptom to cause: localizing errors in counterexample traces”. In: *Proceedings of POPL*. 2003, pp. 97–105.
- [4] B. Liblit et al. “Scalable statistical bug isolation”. In: *PLDI*. 2005.
- [5] M. Weiser. “Program slicing”. In: *Proceedings of ICSE*. 1981, pp. 439–449.
- [6] M. A. Francel and S. Rugaber. “The value of slicing while debugging”. In: *Science of Computer Programming* 40 (2001), pp. 151–169.
- [7] M. Weiser and J. Lyle. “Experiments on slicing-based debugging aids”. In: *Workshop on Empirical Studies of Programmers*. 1986, pp. 187–197.
- [8] Mark Weiser. “Program slicing”. In: *IEEE Transactions on Software Engineering* 10.4 (1984), pp. 352–357.
- [9] Bogdan Korel and Janusz Laski. “Dynamic program slicing”. In: *Information Processing Letters* 29.3 (1988), pp. 155–163.
- [10] Tibor Gyimóthy, Árpád Beszédes, and István Forgács. “An efficient relevant slicing method for debugging”. In: *Proceedings of ESEC/FSE*. 1999, pp. 303–321.
- [11] Xiangyu Zhang, Neelam Gupta, and Rajiv Gupta. “Pruning dynamic slices with confidence”. In: *Proceedings of PLDI*. 2006, pp. 169–180.
- [12] Thomas Reps et al. “The use of program profiling for software maintenance with applications to the year 2000 problem”. In: *Proceedings of ESEC/FSE*. 1997, pp. 432–449.
- [13] Alex Groce, Daniel Kroening, and Flavio Lerda. “Understanding counterexamples with explain”. In: *Proceedings of CAV*. 2004, pp. 453–456.
- [14] James A. Jones, Mary Jean Harrold, and John Stasko. “Visualization of test information to assist fault localization”. In: *Proceedings of ICSE*. 2002, pp. 467–477.
- [15] Manos Renieris and Steven P. Reiss. “Fault localization with nearest neighbor queries”. In: *Proceedings of ASE*. 2003, pp. 30–39.
- [16] Chao Liu and Jiawei Han. “Failure proximity: A fault localization-based approach”. In: *Proceedings of FSE*. 2006, pp. 286–295.
- [17] Dan Hao et al. “A similarity-aware approach to testing based fault localization”. In: *Proceedings of ASE*. 2005, pp. 291–294.
- [18] Shinji Kusumoto et al. “Experimental evaluation of program slicing for fault localization”. In: *Empirical Software Engineering* 7 (2002), pp. 49–76.
- [19] K. D. Sherwood and G. C. Murphy. *Reducing code navigation effort with differential code coverage*. Tech. rep. University of British Columbia, 2008.
- [20] Andrew J. Ko and Brad A. Myers. “Finding causes of program output with the Java Whyline”. In: *Proceedings of CHI*. 2009, pp. 1569–1578.